

# АНАЛИЗ QML КОМПОНЕНТОВ

## НЕОБХОДИМЫЕ КОМПОНЕНТЫ (13 файлов):

### 1. main.qml НУЖЕН

- Главный файл приложения
- Содержит все окна и компоненты
- Статус: ОК, не требует изменений

### 2. SplashScreen.qml НУЖЕН

- Заставка при запуске
- Статус: ОК (размер 800x435 уже исправлен)

### 3. TopMenuBar.qml НУЖЕН

- Верхнее меню (Файл, Правка, Вид...)
- Статус: ОК

### 4. LeftSidebar.qml НУЖЕН

- Панель инструментов слева
- Эффекты, переходы, режимы
- Статус: ОК, подключён к VideoPlayer

### 5. VideoPlayer.qml НУЖЕН

- Просмотр видео
- Статус: ОК, получает эффекты от LeftSidebar

### 6. ModeSwitcher.qml НУЖЕН

- Переключение режимов (Редактирование/Экспорт)
- Статус: ОК

### 7. Timeline.qml НУЖЕН

- Главный компонент timeline
- Содержит дорожки
- Статус:  ТРЕБУЕТ ИСПРАВЛЕНИЯ (см. ниже)

## 8. Track.qml НУЖЕН

- Дорожка на timeline
- Содержит клипы
- Статус:  ТРЕБУЕТ СЕРЬЁЗНОГО ИСПРАВЛЕНИЯ!

## 9. VideoClip.qml НУЖЕН

- Визуальное представление клипа на дорожке
- Статус: ОК

## 10. AudioClip.qml НУЖЕН (но пока не используется)

- Визуализация аудио клипа с waveform
- Статус: ОК, но waveform TODO

## 11. TimeRuler.qml НУЖЕН

- Линейка времени сверху
- Статус: ОК

## 12. PlaybackControls.qml НУЖЕН

- Кнопки Play/Pause, перемотка
- Статус: ОК

## 13. ClipManager.qml НУЖЕН

- Менеджер клипов (логика)
- Статус:  КРИТИЧНО! Содержит mock код!

---

## ИЗБЫТОЧНЫЕ КОМПОНЕНТЫ:

### НЕТ ИЗБЫТОЧНЫХ!

Все 13 компонентов необходимы для работы редактора.

---

## ЧТО НУЖНО ИСПРАВИТЬ:

### КРИТИЧНО: ClipManager.qml

Проблема:

qml

// ТЕКУЩИЙ КОД (НЕПРАВИЛЬНО):

```
function addClip(filepath, trackNumber, startTime, duration) {
  var clipId = clips.length
  var mockDuration = duration || (Math.random() * 50 + 10) // ← МОСК!

  var clip = {
    id: clipId,
    filepath: filepath,
    startTime: startTime,
    duration: mockDuration, // ← ФЕЙК ДЛИТЕЛЬНОСТЬ!
    filename: filepath.split('/').pop()
  }

  clips.push(clip) // ← Добавляет в QML массив вместо C++!
  revision++
}
```

### Решение:

qml

// ПРАВИЛЬНЫЙ КОД:

```
function addClip(filepath, trackNumber, startTime) {
  // Проверка расширения
  var ext = filepath.split('.').pop().toLowerCase()
  var videoExts = ['mp4', 'avi', 'mov', 'mkv', 'webm']
  if (videoExts.indexOf(ext) === -1) {
    console.log("❌ Не видео файл")
    return
  }

  // Вызвать C++!
  var success = cppTimeline.addClip(filepath, trackNumber, startTime)

  // C++ сам emit clipsChanged() → Track обновится!
}
```

### Что удалить:

- ❌ `property var clips: []` - не нужен, clips в C++!
- ❌ `property int revision: 0` - не нужен
- ❌ Всю mock логику генерации данных

## ❌ КРИТИЧНО: Track.qml

### Проблема:

```
qml

// ТЕКУЩИЙ КОД (НЕПРАВИЛЬНО):
property var clips: [] // Mock clips!

Repeater {
    model: root.clips // ← МОСК данные!

    delegate: Loader {
        // Создаёт VideoClip с НЕПРАВИЛЬНЫМИ данными
    }
}
```

### Решение:

```
qml
```

```

// ПРАВИЛЬНЫЙ КОД:
property var realClips: [] // Реальные из C++

// Функция обновления
function updateClipsFromCpp() {
    realClips = cppTimeline.getClipsForTrack(trackNumber)
}

// Слушаем изменения
Connections {
    target: cppTimeline
    function onClipsChanged() {
        updateClipsFromCpp() // ← Обновляем!
    }
}

// Инициализация
Component.onCompleted: {
    updateClipsFromCpp()
}

// Repeater
Repeater {
    model: root.realClips // ← РЕАЛЬНЫЕ из C++!

    delegate: Loader {
        onLoaded: {
            var clipData = modelData // ← Данные из C++
            item.clipName = clipData.filename
            item.x = clipData.startTime * pixelsPerSecond
            item.width = clipData.duration * pixelsPerSecond // ← РЕАЛЬНАЯ!
        }
    }
}

```

### Что добавить:

- ✓ `function updateClipsFromCpp()`
- ✓ `Connections { target: cppTimeline }`
- ✓ Вызов `updateClipsFromCpp()` в `Component.onCompleted`

### ⚠ Timeline.qml

### Проблема:

```
qml
```

```
// Где-то есть обращение к clipManager.clips
```

```
// Но clips теперь в C++, не в QML!
```

**Решение:** Найти все места где используется `clipManager.clips` и заменить на вызовы C++:

```
qml
```

```
// БЫЛО:
```

```
var duration = 0
```

```
for (var i = 0; i < clipManager.clips.length; i++) {
```

```
    duration = Math.max(duration, clipManager.clips[i].endTime)
```

```
}
```

```
// СТАЛО:
```

```
var duration = cppTimeline.totalDuration // ← Q_PROPERTY!
```

## ПОШАГОВЫЙ ПЛАН ИСПРАВЛЕНИЯ:

### Шаг 1: C++ Backend

- ✓ 1. Добавь в timeline.h:
  - #include <QVariantList>
  - Q\_INVOKABLE QVariantList getClipsForTrack(int)
- ✓ 2. Добавь в timeline.cpp:
  - Реализацию getClipsForTrack()

### Шаг 2: ClipManager.qml

- ✓ 1. Удали: property var clips: []
- ✓ 2. Удали: property int revision: 0
- ✓ 3. Удали: всю mock логику из addClip()
- ✓ 4. Добавь: вызов cppTimeline.addClip()
- ✓ 5. Добавь: проверку расширения файла
- ✓ 6. Добавь: Connections для clipsChanged

### Шаг 3: Track.qml

- ✓ 1. Замени: property var clips: [] → property var realClips: []
- ✓ 2. Добавь: function updateClipsFromCpp()
- ✓ 3. Добавь: Connections { target: cppTimeline }

- ✓ 4. Добавь: `Component.onCompleted { updateClipsFromCpp() }`
- ✓ 5. Измени: `Repeater model: realClips`
- ✓ 6. Исправь: `delegate` использует `modelData` из C++

#### Шаг 4: Timeline.qml (если нужно)

- ⚠ 1. Найди все `clipManager.clips`
- ⚠ 2. Замени на `cppTimeline.totalDuration` или другие `Q_PROPERTY`

### ИТОГОВАЯ СТАТИСТИКА:

#### Компоненты:

- ✓ Всего: 13 файлов
- ✓ Необходимых: 13 (100%)
- ✗ Избыточных: 0

#### Состояние:

- ✓ ОК: 10 файлов (77%)
- ⚠ Требуется исправления: 2 файла (15%)
- ✗ Критично: 1 файл (8%)

#### Критичные файлы:

1. ✗ **ClipManager.qml** - содержит `mock`, НЕ РАБОТАЕТ с C++
2. ⚠ **Track.qml** - не синхронизирован с C++
3. ⚠ **Timeline.qml** - возможно использует `clipManager.clips`

### ✓ ПОСЛЕ ИСПРАВЛЕНИЯ:

#### БЫЛО:

User drag video.mp4  
↓  
ClipManager создаёт `mock clip` (10 сек) ← НЕПРАВИЛЬНО!  
↓  
Track показывает `mock` (100 px) ← НЕПРАВИЛЬНО!

#### СТАЛО:

User drag video.mp4



ClipManager → cppTimeline.addClip() ← C++!



C++ → MediaDecoder.getDuration() → 71.6 сек ← РЕАЛЬНО!



C++ → emit clipsChanged()



Track → updateClipsFromCpp()



Track → cppTimeline.getClipsForTrack()



Track → Repeater создаёт VideoClip (716 px) ← ПРАВИЛЬНО!



## ПРЕВЬЮ КАДРОВ (TODO):

Ты упомянул превью - это отдельная фича. Нужно:

cpp

// В Timeline.h добавить:

Q\_INVOKABLE void generateThumbnail(int clipIndex, double time);

signals:

void thumbnailReady(int clipIndex, QString thumbnailPath);

// В Timeline.cpp:

void Timeline::generateThumbnail(int clipIndex, double time) {

if (clipIndex >= m\_clips.size()) return;

MediaDecoder decoder;

decoder.openFile(m\_clips[clipIndex].filepath);

QImage frame = decoder.getFrameAt(time);

QString thumbPath = QDir::temp().filePath("thumb\_" + QString::number(clipIndex) + ".png");

frame.save(thumbPath);

emit thumbnailReady(clipIndex, thumbPath);

}

qml



```
// B VideoClip.qml:
Component.onCompleted: {
    cppTimeline.generateThumbnail(clipId, 0.0)
}

Connections {
    target: cppTimeline
    function onThumbnailReady(clipIndex, path) {
        if (clipIndex === root.clipId) {
            root.thumbnailPath = path
        }
    }
}
```

**Но это ПОСЛЕ того как основное работает!**

---

**ИСПОЛЬЗУЙ ГОТОВЫЕ ФАЙЛЫ КОТОРЫЕ Я СОЗДАЛ! 🚀**

**timeline\_fixed.h, timeline\_fixed.cpp, ClipManager\_fixed.qml, Track\_fixed.qml**